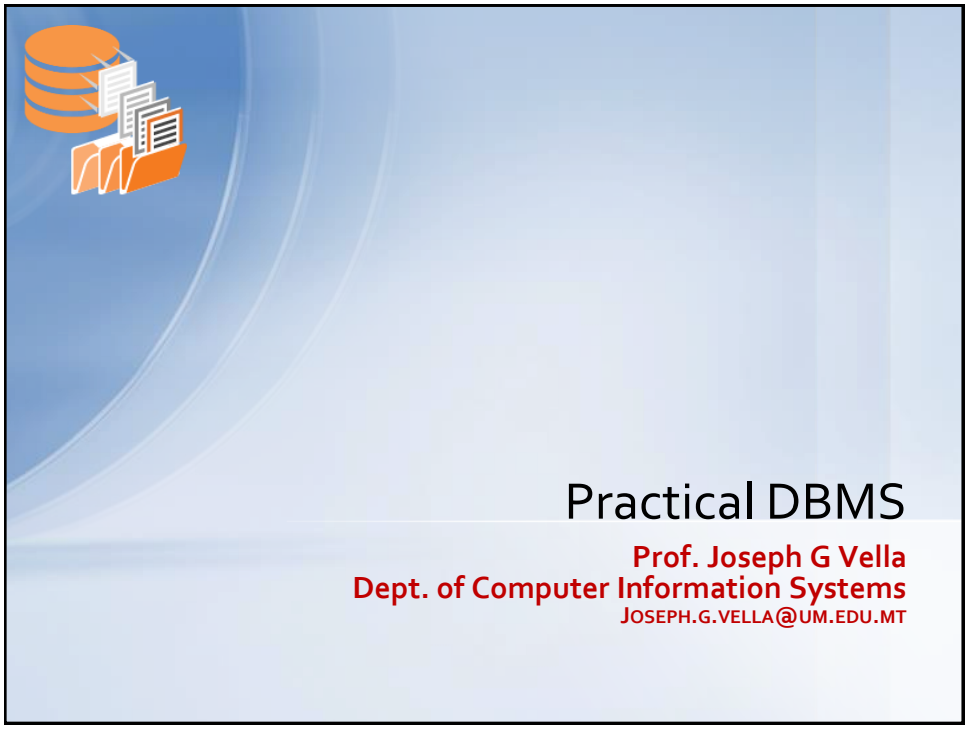
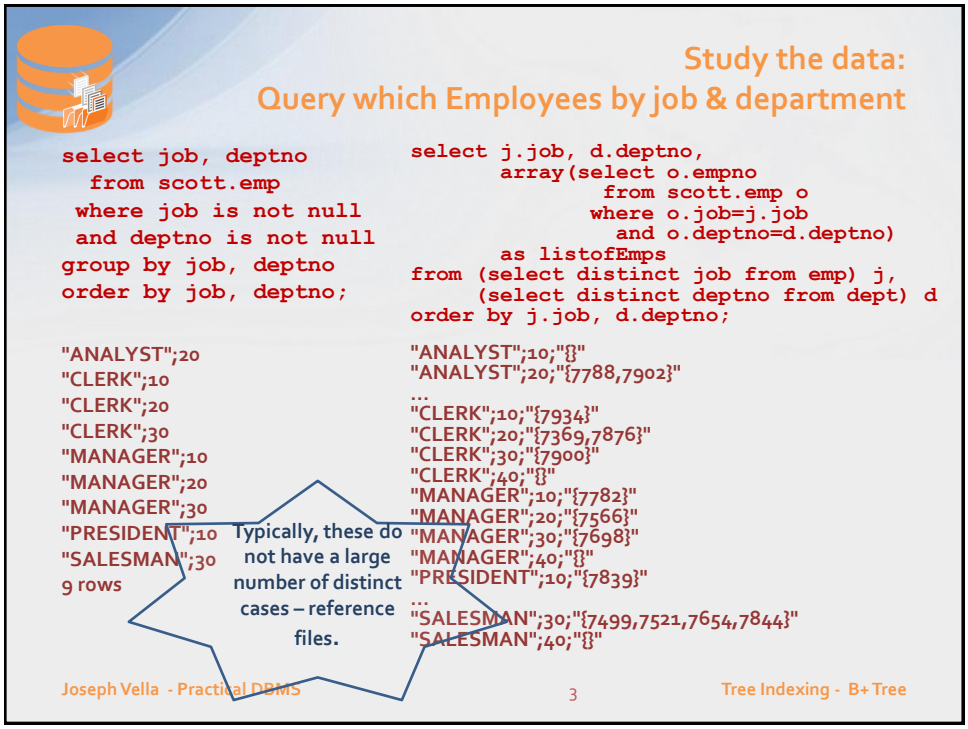

Practical DBMS
Prof. Joseph G Vella
Dept. of Computer Information Systems
JOSEPH.G.VELLA@UM.EDU.MT

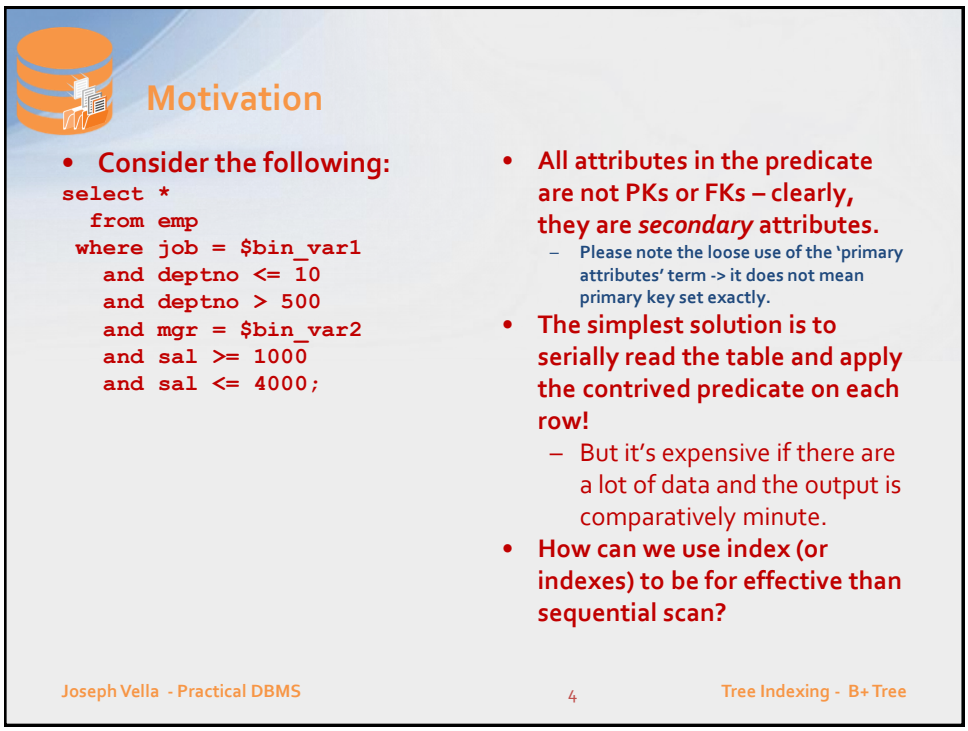
1


Practical DBMS
Indexing: Bitmap
(formally called an Inverted Index)
– a term is also of great interest in Information Retrieval

2


Study the data:
Query which Employees by job & department
select job, deptno
from scott.emp
where job is not null
and deptno is not null
group by job, deptno
order by job, deptno;
"ANALYST";20
"CLERK";10
"CLERK";20
"CLERK";30
"MANAGER";10
"MANAGER";20
"MANAGER";30
"PRESIDENT";10
"SALESMAN";30
9 rows
Typically, these do not have a large number of distinct cases – reference files.
select j.job, d.deptno,
array(select o.empno
from scott.emp o
where o.job=j.job
and o.deptno=d.deptno)
as listofEmps
from (select distinct job from emp) j,
(select distinct deptno from dept) d
order by j.job, d.deptno;
"ANALYST";10;"{}"
"ANALYST";20;"{7788,7902}"
...
"CLERK";10;"{7934}"
"CLERK";20;"{7369,7876}"
"CLERK";30;"{7900}"
"CLERK";40;"{}"
"MANAGER";10;"{7782}"
"MANAGER";20;"{7566}"
"MANAGER";30;"{7698}"
"MANAGER";40;"{}"
"PRESIDENT";10;"{7839}"
...
"SALESMAN";30;"{7499,7521,7654,7844}"
"SALESMAN";40;"{}"
Joseph Vella - Practical DBMS
3
Tree Indexing - B+ Tree

3


Motivation
Consider the following:
select *
from emp
where job = $bin_var1
and deptno <= 10
and deptno > 500
and mgr = $bin_var2
and sal >= 1000
and sal <= 4000;
All attributes in the predicate are not PKs or FKs – clearly, they are secondary attributes.
Please note the loose use of the 'primary attributes' term -> it does not mean primary key set exactly.
The simplest solution is to serially read the table and apply the contrived predicate on each row!
But it's expensive if there are a lot of data and the output is comparatively minute.
How can we use index (or indexes) to be for effective than sequential scan?
Joseph Vella - Practical DBMS
4
Tree Indexing - B+ Tree

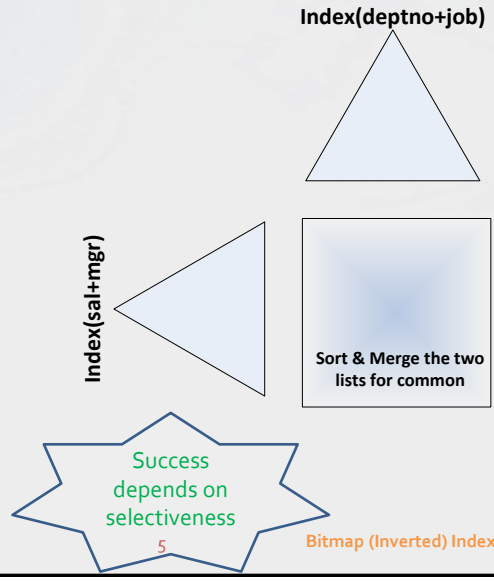
4

## Solution One:

- **Restating query:**

```
select *
  from emp
 where job = $bin_var1
   and deptno <= 10
   and deptno > 500
   and mgr = $bin_var2
   and sal >= 1000
   and sal <= 4000;
```

- **Use compound indexes (ordered):**
  - Index(deptno+job);
  - Index(sal+mgr);
- **Note that these are useless here:**
  - Index(job+deptno); or
  - Index (mgr+sal).





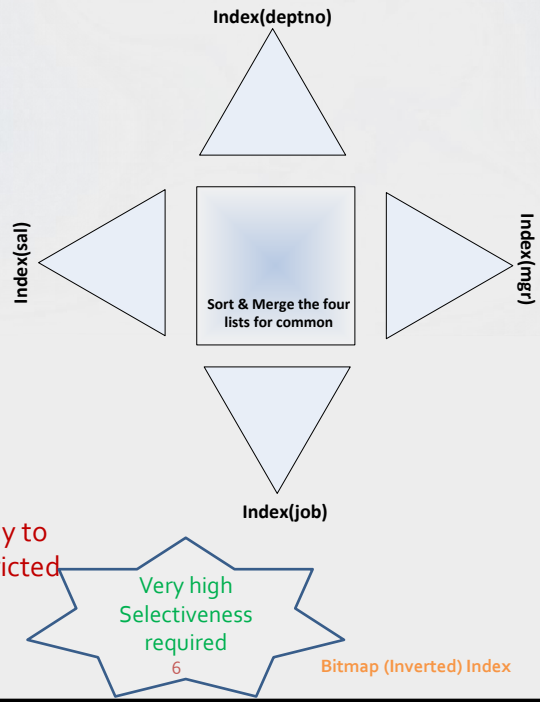

5

## Solution Two:

- **Restating query:**

```
select *
  from emp
 where job = $bin_var1
   and deptno <= 10
   and deptno > 500
   and mgr = $bin_var2
   and sal >= 1000
   and sal <= 4000;
```

- **Tactic is to index every attribute!**
  - Maintenance is high;
  - Range queries unlikely to be selective (i.e. restricted range).






6

## Bitmaps

- **If an access request is not restricted to one primary key attribute, simple direct-access methods fail.**
  - *For example:*
    Return personnel records that satisfy these conditions
    - **in the 'programming department'**
    - **can speak in 'French'**
    - **not in grade '3' unless from the 'North branch'**
- **Bitmaps (or Inverted files in older literature) are an example of a *secondary index file structure*.**
  - *Bitmaps convert search keys, from different attributes, into a bit sequence related either to a record or to an attribute instance.*
  - A secondary index file structure is a table for relating *secondary key field values* to a number of master file records that have that value.
    - **These are essential for querying on:**
      - **Multiple attributes;**
      - **Sequence of Joins.**



7

## More detailed example

| *Master file key* | *Name* | *Sex* | *Grade* | *Department* | *Branch* | *Languages spoken* |
|---|---|---|---|---|---|---|
| 18 | Brown | M | 1 | Systems | South | — |
| 23 | Collins | M | 3 | Prog | North | — |
| 35 | Duncan | F | 2 | Prog | North | French |
| 41 | Farrar | M | 5 | Admin | Central | — |
| 42 | Jablonski | F | 4 | Operations | North | — |
| 47 | Kellman | F | 3 | Systems | North | German. Polish |
| 59 | Lawrence | M | 3 | Prog | Central | — |
| 83 | Mason | F | 2 | Systems | South | — |
| 94 | Norris | M | 1 | Operations | South | — |
| 101 | Oliver | F | 1 | Operations | Central | French. German |
| 110 | Ordinale | F | 4 | Systems | South | — |
| 115 | Stewart | F | 3 | Prog | North | — |
| 122 | Taylor | M | 1 | Operations | Central | French. German. Italian |
| 128 | Vince | F | 2 | Systems | South | — |
| 133 | Westcott | M | 1 | Prog | South | Italian |



8

| Class | Attribute | Master file keys |
|---|---|---|
| Sex | M | 18, 23, 41, 59, 94, 122, 133 |
| | F | 35, 42, 47, 83, 101, 110, 115, 128 |
| Grade | 1 | 18, 94, 101, 122, 133 |
| | 2 | 35, 83, 128 |
| | 3 | 23, 47, 59, 115 |
| | 4 | 42, 110 |
| | 5 | 41 |
| Department | Systems | 18, 47, 83, 110, 128 |
| | Prog | 23, 35, 59, 115, 133 |
| | Admin | 41, |
| | Operations | 42, 94, 101, 122 |
| Branch | South | 18, 83, 94, 110, 128, 133 |
| | North | 23, 35, 42, 47, 115 |
| | Central | 41, 59, 101, 122 |
| Languages spoken | French | 35, 101, 122 |
| | German | 47, 101, 122 |
| | Polish | 47 |
| | Italian | 122, 133 |

continued



9

## Bitmaps Variety

- **A file that inverts all the attributes of a data file into a Bitmap is known as a *fully inverted index*.**
  - Some implementations of fully inverted files make the data file on-line retention redundant!
- **A file structure that inverts only a subset of the data file attributes is known as a *partially inverted index*.**
  - Again, some implementation can mask out from the data file those attributes that are inverted!



10

## Bitmaps detailed example: earlier case converted to bits

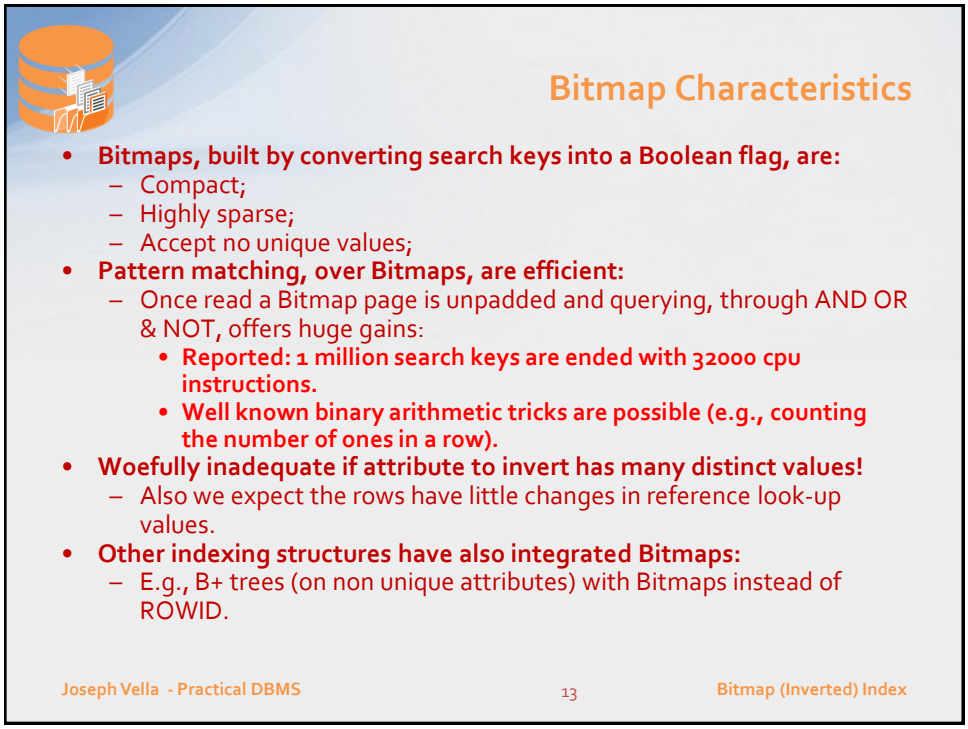
| Master file key | Name | Sex | Grade | Department | Branch | Languages spoken |
|---|---|---|---|---|---|---|
| 18 | Brown | M | 1 | Systems | South | — |
| 23 | Collins | M | 3 | Prog | North | — |
| 35 | Duncan | F | 2 | Prog | North | French |
| 41 | Farrar | M | 5 | Admin | Central | — |
| 42 | Jablonski | F | 4 | Operations | North | — |
| 47 | Kellman | F | 3 | Systems | North | German, Polish |
| 59 | Lawrence | M | 3 | Prog | | |
| 83 | Mason | F | 2 | Systems | | |
| 94 | Norris | M | 1 | Operatic | | |
| 101 | Oliver | F | 1 | Operatic | | |
| 110 | Ordinale | F | 4 | Systems | | |
| 115 | Stewart | F | 3 | Prog | | |
| 122 | Taylor | M | 1 | Operatic | | |
| 128 | Vince | F | 2 | Systems | | |
| 133 | Westcott | M | 1 | Prog | | |

| Name | | Brown | Collins | Duncan | Farrar | Jablonski | Kellman | Lawrence | Mason | Norris | Oliver | Ordinale | Stewart | Taylor | Vince | Westcott |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Master file key | | 18 | 23 | 35 | 41 | 42 | 47 | 59 | 83 | 94 | 101 | 110 | 115 | 122 | 128 | 133 |
| Sex | M | 1 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 1 |
| | F | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 0 |
| Grade | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 1 |
| | 2 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| | 3 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| | 4 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| | 5 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Department | Systems | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 |
| | Prog | 0 | 1 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 |
| | Admin | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| | Operations | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| Branch | South | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 1 | 1 |
| | North | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| | Central | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| Language | French | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| | German | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| | Polish | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| | Italian | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 |

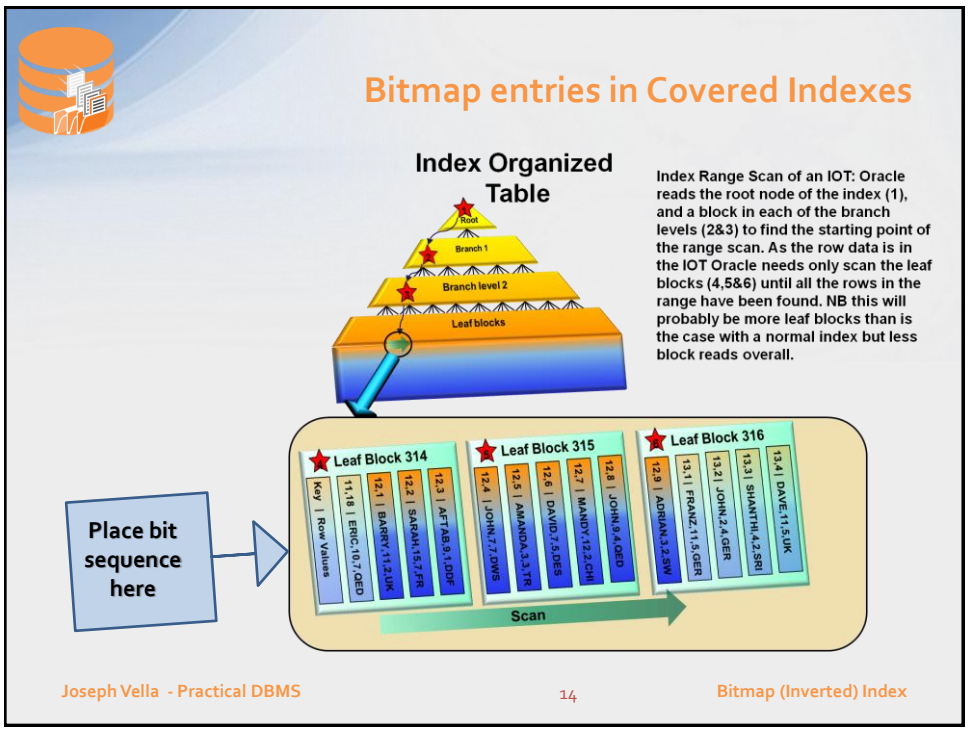


## Inverting Technique – Building a Bitmap

- ***Every data value in a record's attribute is represented as a key and the supplementary value in the index is a list of records pointers (to the data file) that hold that value.***
  ***This is, in essence, the technique of inverting*** **(some references use inverting in a more general connotations - i.e. that of any indexing procedures).**
- ***When*, *How*,** and on ***What*** do we invert?
  - Delicate question!
    What type of referencing should we introduce and when do we pin to actual physical records?


Bitmap Characteristics
• Bitmaps, built by converting search keys into a Boolean flag, are:
– Compact;
– Highly sparse;
– Accept no unique values;
• Pattern matching, over Bitmaps, are efficient:
– Once read a Bitmap page is unpadded and querying, through AND OR & NOT, offers huge gains:
• Reported: 1 million search keys are ended with 32000 cpu instructions.
• Well known binary arithmetic tricks are possible (e.g., counting the number of ones in a row).
• Woefully inadequate if attribute to invert has many distinct values!
– Also we expect the rows have little changes in reference look-up values.
• Other indexing structures have also integrated Bitmaps:
– E.g., B+ trees (on non unique attributes) with Bitmaps instead of ROWID.
Joseph Vella - Practical DBMS
13
Bitmap (Inverted) Index

13


Bitmap entries in Covered Indexes
Index Organized Table
Root
Branch 1
Branch level 2
Leaf blocks
Index Range Scan of an IOT: Oracle reads the root node of the index (1), and a block in each of the branch levels (2&3) to find the starting point of the range scan. As the row data is in the IOT Oracle needs only scan the leaf blocks (4,5&6) until all the rows in the range have been found. NB this will probably be more leaf blocks than is the case with a normal index but less block reads overall.
Place bit sequence here
Leaf Block 314
Key | Row Values
11,18 | ERIC,10,7,QED
12,1 | BARRY,11,2,UK
12,2 | SARAH,15,7,FR
12,3 | AFTAB,9,1,DDF
Leaf Block 315
12,4 | JOHN,7,7,DWS
12,5 | AMANDA,3,3,TR
12,6 | DAVID,7,5,DES
12,7 | MANDY,12,2,CHI
12,8 | JOHN,4,4,QED
Leaf Block 316
12,9 | ADRIAN,3,2,SW
13,1 | FRANZ,11,5,GER
13,2 | JOHN,2,4,GER
13,3 | SHANTHI,4,2,SRI
13,4 | DAVE,11,5,UK
Scan
Joseph Vella - Practical DBMS
14
Bitmap (Inverted) Index

14

## Bitmap & Oracle

```
CREATE TABLE scott.emp(
  empno     integer
            NOT NULL RIMARY KEY,
  ename     character var (10)
            NOT NULL,
  job       character var(9),
  mgr       integer,
  hiredate  date,
  sal       numeric,
  comm      numeric,
  deptno    integer);
```

- **Candidate attributes for inverting are:**
  - **job, mgr, deptno;**
  - **hiredate** is also one, but to reduce cardinality it's best to invert into a year;

```
• CREATE BITMAP INDEX
  emp_inv_job(job);
  CREATE BITMAP INDEX
  emp_inv_mgr (mgr);
  CREATE BITMAP INDEX
  emp_inv_deptno(deptno);
```

- **Oracle Syntax (since 7.3 – circa 2000):**
  - Query optimiser will favour the following type of queries:

```
• SELECT *
  FROM emp
  WHERE job='MANAGER'
  AND deptno=20;
```



15

## Bitmap Indices Access Methods

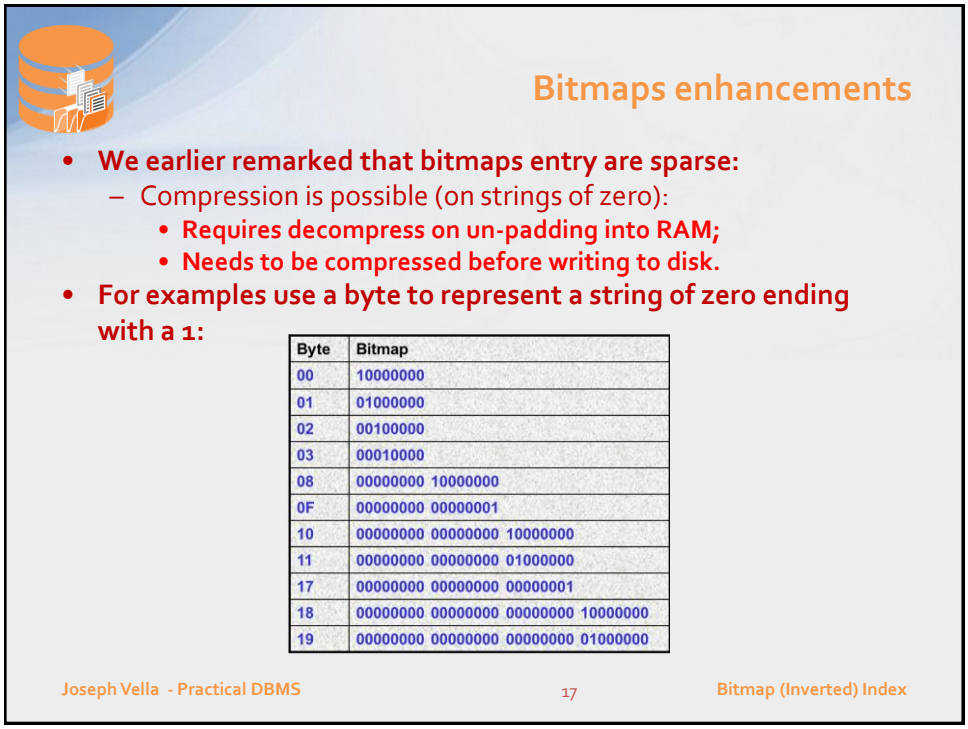
- **Queries that are addressed by Bitmaps include:**
  - AND as intersection
  - OR as union
  - NOT as complementation
  - MINUS
    - **A AND NOT B**

| A | B | A AND B | A OR B | NOT A |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 0 |

- **Each operation takes two bitmaps of the same size and applies the operation on corresponding bits to get the result bitmap**
  - E.g. 100110 AND 110011 = 100010
    100110 OR 110011 = 110111
    NOT 100110 = 011001
  - Males with income level L1: 10010 AND 10100 = 10000
    - **Can then retrieve required tuples.**
    - **Counting number of matching tuples is even faster.**
- **Bitmap re-orginisation, e.g. on DELETEs, is a major operation.**



16

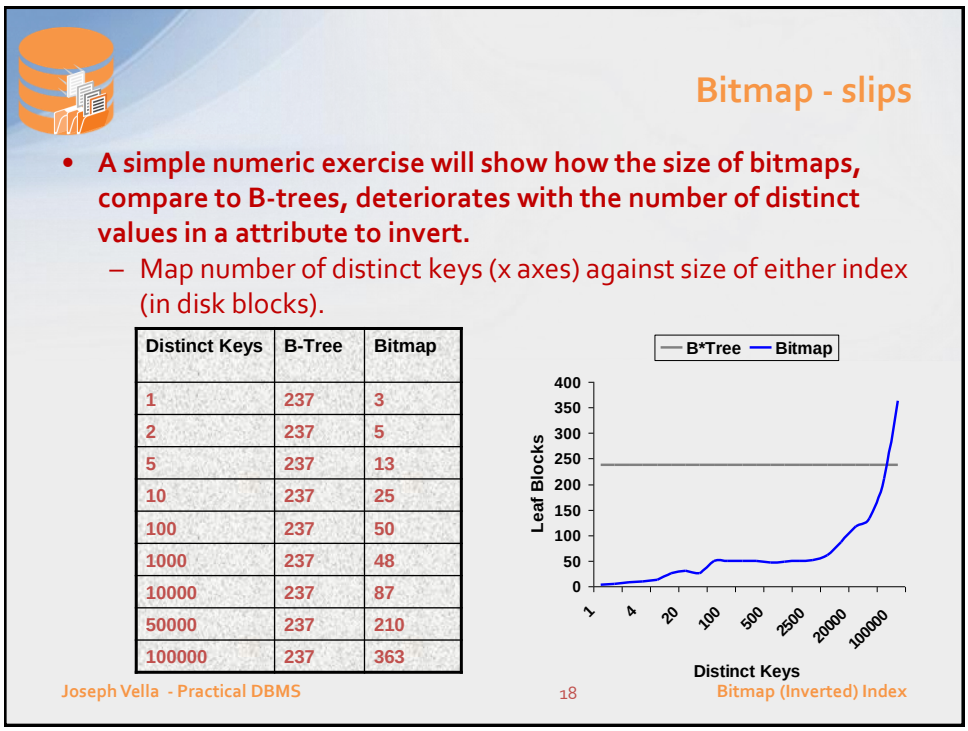


## Bitmaps enhancements

- We earlier remarked that bitmaps entry are sparse:
  - Compression is possible (on strings of zero):
    - Requires decompress on un-padding into RAM;
    - Needs to be compressed before writing to disk.
- For examples use a byte to represent a string of zero ending with a 1:

| Byte | Bitmap |
|---|---|
| 00 | 10000000 |
| 01 | 01000000 |
| 02 | 00100000 |
| 03 | 00010000 |
| 08 | 00000000 10000000 |
| 0F | 00000000 00000001 |
| 10 | 00000000 00000000 10000000 |
| 11 | 00000000 00000000 01000000 |
| 17 | 00000000 00000000 00000001 |
| 18 | 00000000 00000000 00000000 10000000 |
| 19 | 00000000 00000000 00000000 01000000 |





## Bitmap - slips

- A simple numeric exercise will show how the size of bitmaps, compare to B-trees, deteriorates with the number of distinct values in a attribute to invert.
  - Map number of distinct keys (x axes) against size of either index (in disk blocks).

| Distinct Keys | B-Tree | Bitmap |
|---|---|---|
| 1 | 237 | 3 |
| 2 | 237 | 5 |
| 5 | 237 | 13 |
| 10 | 237 | 25 |
| 100 | 237 | 50 |
| 1000 | 237 | 48 |
| 10000 | 237 | 87 |
| 50000 | 237 | 210 |
| 100000 | 237 | 363 |

## Support possibilities to Secondary Indices

- **Other file structures that support secondary index demands include**
  - *bit pattern indexes*;
  - *a collection of B trees* based on sub strings of concatenated key strings; and
  - advanced data structures like Grid files and R-Trees (these are classified as multi-dimensional indexes).



19

## Choosing Bitmaps

- **Which to choose? This is strongly influenced by the following parameters:**
  - speed / access requirements;
  - volumetrics of the master file;
  - secondary key filed size;
  - secondary key field data distribution;
  - resilience requirement.



20